

Execution-Time Opacity Logic: A Logic for Ensuring ET-Opacity in Timed Systems

Jean Leneutre³ , Dylan Marinho² , Vadim Malvone³ , and James Ortiz¹ 

¹ Université Paris Est Créteil, LACL, Créteil, France

² Sorbonne Université, CNRS UMR 7606, LIP6, Paris, France

³ LTCI, Télécom Paris, Institut Polytechnique de Paris, Palaiseau, France

Abstract. Ensuring confidentiality in Cyber-Physical Systems is critical, especially when attackers exploit execution times to infer sensitive information. Traditional opacity models are inadequate for timed systems, as verifying opacity in Timed Automata is undecidable. To address this challenge, we propose *Execution-Time Opacity Logic* (ETOL), a new formalism that specifies opacity by requiring that for every execution satisfying a secret formula, there exists another execution of the same duration that does not satisfy it. ETOL guarantees that timing observations cannot reveal confidential agent activities. We present a decidable and efficient verification framework based on zone-based model checking, supported by a dedicated algorithm that systematically identifies duration-equivalent executions. Our approach is validated through an ATM case study, showing that ETOL enables efficient verification of execution-time confidentiality under timing attacks. We also developed a prototype tool for the ETOL logic that supports symbolic model checking over timed systems. It allows users to verify ETOL formulas based on clock-constrained execution paths.

1 Introduction

In the modern digital age, the increasing complexity of distributed infrastructures and *Cyber-Physical Systems* has introduced new verification challenges in cybersecurity. Critical systems such as those in healthcare, transportation, and industrial control must operate under strict time constraints while preserving the confidentiality of sensitive information. Cybersecurity is a key domain where this need becomes particularly acute. As attackers grow more sophisticated, it is essential not only to defend against direct attacks but also to ensure that confidential data remains hidden even through system observations. This security property is known as *opacity* [23,19,16], where it refers to the inability of an external observer to deduce secret information from event sequences. In critical *Real-Time Systems* (RTSs), however, the timing of events introduces a new threat dimension: attackers may exploit execution durations to infer secrets that would otherwise remain hidden. This has led to the development of *timed opacity*, which studies how timing observations can compromise confidentiality.

A challenge in RTSs is that attacks are not limited to observing system states and their timestamps, but may also leverage execution times themselves to extract secrets [24]. For example, side-channel attacks use the execution time of specific operations to infer secret information, such as cryptographic keys, by measuring the time differences between two operations. In such attacks, even if sensitive information is kept encrypted by traditional defense mechanisms, it can be extracted by attackers by the time operations being analysed. For example, Brumley and Boneh demonstrated in [13] that an attacker could recover a 1024-bit RSA private key from an OpenSSL server by measuring network response times, using approximately one million queries over about two hours.

Timed opacity for *Timed Automata* (TAs) [1] was first introduced by Cassez [17], defining it as the inability to deduce secret actions based on an observation of a timed trace. However, he showed that verifying timed opacity for a general TA is undecidable, even in (very) restricted subclasses, due to the complexity of the timed language inclusion problem. Subsequent works proposed partial solutions.

Restricting the model Some approaches limit the expressiveness of the system model to regain decidability. For instance, [4] identified decidability in certain contexts, such as one-clock TA without ϵ -transitions. Others focus on subclasses of TA, like *Real-Time Automata*⁴, and restrict the notion of opacity to initial state opacity [30].

Restricting the notion of opacity Some works restrict the notion of opacity itself, for example by considering only *initial-state opacity* [30], which asks whether an observer can infer that the execution started from a secret initial state (or, more generally, from a secret set of initial states). This restriction often simplifies the verification problem, but it may fail to capture richer confidentiality properties arising from secrets revealed only later during the execution.

Restricting the attacker’s capabilities Another approach is to limit the attacker’s observational power, for instance by bounding the time window within which observations can be made or compared [2]. Such restrictions often make the verification problem more tractable, although they may also weaken the attacker model.

Restricting the observation model Instead of using traditional observability functions, which often lead to undecidability, some approaches adopt alternative observation models. For example, [6] introduces a model based on *duration observations* defining *Execution-Time Opacity* (ET-opacity), where the observer measures only the total execution time to a final state, without access to trace events. This model brings decidability while addressing realistic side-channel threats, as execution times can be a sufficient source of information leakage.

However, all of these approaches focus on location-based opacity, where the secret is defined as the visit of a particular control location. We argue that this definition is too restrictive for many real-world scenarios, where secrets may depend on complex combinations of control states and timing conditions.

⁴ which can be seen as a subclass of TAs with a single clock, reset at each transition

Our contribution. We introduce ETOL, a branching-time temporal logic for specifying and verifying ET-opacity properties with timing constraints. We define the syntax and semantics of ETOL, and enrich its branching-time framework with two dedicated modalities: $\exists$ and $\forall$, which quantify over runs that are indistinguishable to an attacker based on their total execution time. Principally, ETOL allows us to express that a secret property (i.e. its satisfaction) is not revealed to an attacker by the total execution time. From a theoretical perspective, we propose a PSPACE symbolic procedure for model checking ETOL over a TA.

Related work. In addition to the works on timed opacity mentioned above, there are several related lines of research. Related real-time security lines focus on non-interference: [26] proposes a type system handling nondeterminism and timing, [28] defines information-flow-based bisimulation and computes local security constraints, and [21] reduces verification to unreachability checked with UPPAAL [8]. Control synthesis for strong non-deterministic non-interference in TA is studied in [9]. Recently, [29] extends PTCTL (a parametric extension of *Timed Computation Tree Logic* (TCTL) [1]) with hyper-properties, allowing to express properties over multiple runs. Their models and results are not really comparable to ours, as they focus on parametric hyper-properties, while we focus on a specific class of hyper-properties related to execution time without any parameter. In particular, there is no inclusion between our logics ETOL and their logic, and neither logic subsumes the other. More broadly, opacity has been extensively studied in the setting of Discrete-Event Systems (DES), using both automata and Petri nets. Classical language-based notions include *initial-state opacity*, where the observer should not be able to infer that the system started from a secret initial state, *current-state opacity*, where the observer should not be able to determine that the current state is secret, and *k-step opacity*, where secrecy must remain hidden over a bounded observation horizon [25,14,15]. These notions form the standard semantic basis of opacity. They have given rise to a variety of verification and enforcement techniques. Automata-based opacity verifiers decide whether observations remain compatible with both secret and non-secret behaviours [27], while Petri net-based analyses address concurrent systems where synchronization is essential [14]. These works provide the general conceptual background of opacity, whereas our contribution focuses on execution-time opacity in timed systems and on its logical and symbolic verification.

Outline. The paper is structured as follows. Section 2 recalls the necessary theoretical background. Section 3 introduces the syntax and semantics of ETOL, with the extension of the definition of ET-opacity, and illustrates its expressiveness with examples. In Section 4, we present our model checking algorithm for ETOL over TA, prove its correctness, and establish its PSPACE complexity. The implementation of our model checking procedure and its evaluation on a case study of an ATM system are described in Section 5. Finally, we conclude in Section 6 and discuss future research directions. Due to space limitations, proofs have been moved to the appendix.

2 Preliminaries

In this section, we introduce the core notations and formalisms. Let $\mathbb{R}_{\geq 0}, \mathbb{Z}, \mathbb{N}_{\geq 0}$ be the sets of non-negative reals, integers, and positive integers, respectively. Given two sets $\mathbb{X}$, let $|\mathbb{X}|$ denote the cardinality of $\mathbb{X}$. Given $\mathbb{X}$ and $\mathbb{Y}$, the set operations of intersection, union, set difference, and Cartesian product are denoted $\mathbb{X} \cap \mathbb{Y}, \mathbb{X} \cup \mathbb{Y}, \mathbb{X} \setminus \mathbb{Y}$, and $\mathbb{X} \times \mathbb{Y}$, respectively. Inclusion and strict inclusion are denoted $\mathbb{X} \subseteq \mathbb{Y}$ and $\mathbb{X} \subset \mathbb{Y}$, respectively. The empty set is denoted $\emptyset$.

We assume a set $\mathbb{C} = \{x_1, \dots, x_H\}$ of *clocks*, i.e. real-valued variables that all evolve over time at the same rate. A clock valuation is a function $\nu : \mathbb{C} \rightarrow \mathbb{R}_{\geq 0}$. We write $\mathbf{0}$ for the clock valuation assigning 0 to all clocks. Given $d \in \mathbb{R}_{\geq 0}, \nu + d$ denotes the valuation s.t. $(\nu + d)(x) = \nu(x) + d$, for all $x \in \mathbb{C}$. Given $R \subseteq \mathbb{C}$, we define the *reset* of a valuation ν , $[\nu]_R$, as follows: $[\nu]_R(x) = 0$ if $x \in R$, and $[\nu]_R(x) = \nu(x)$ otherwise. A *clock guard* g is a conjunction of constraints over $\mathbb{C}$ of the form $x \bowtie d$, where $x \in \mathbb{C}$, $d \in \mathbb{Z}$, and $\bowtie \in \{<, \leq, =, \geq, >\}$. The set of all clock guards is denoted $\mathbb{G}(\mathbb{C})$. Given g , we write $\nu \models g$ if the expression obtained by replacing each x with $\nu(x)$ in g evaluates to true.

2.1 Timed Automata and their Semantics

We now recall the definition of TAs, which are an extension of *Finite Automata* with a finite set of clocks that evolve synchronously with time and allow measurement of delays.

Definition 1 (Timed Automata [1]). A TA is a tuple $\mathcal{A} = (\mathbb{L}, \ell_0, \mathbb{C}, \Sigma, \text{Inv}, \text{Lab}, \mathbb{F}, \mathbb{T})$, where: 1. $\mathbb{L}$ is a finite set of locations, 2. $\ell_0 \in \mathbb{L}$ is an initial location, 3. $\mathbb{C}$ is a finite set of clocks, 4. Σ is a finite set of actions, 5. Inv is a function that associates to each location clock guard in which $\bowtie \in \{<, \leq\}$ (called invariant), 6. $\text{Lab} : \mathbb{L} \rightarrow 2^{AP}$ is a labeling function for the locations (AP is a set of atomic propositions), 7. $\mathbb{F} \subseteq \mathbb{L}$ is a set of final locations. 8. $\mathbb{T} \subseteq \mathbb{L} \times \Sigma \times \mathbb{G}(\mathbb{C}) \times 2^{\mathbb{C}} \times \mathbb{L}$ is a finite set of transitions.

A transition $e = (\ell_1, a, g, R, \ell_2) \in \mathbb{T}$ can be written as $\ell_1 \xrightarrow{a, g, R} \ell_2$ where ℓ_1 and ℓ_2 are the source and target locations, a the action, g a guard, R the set of clocks to reset. To define the semantics of a TA, we recall the definition of a *Timed Transition System* (TTS). A TTS is a tuple $(S, s_0, S_F, \Sigma, \text{Lab}_{\mathcal{T}}, E)$ where 1. S is a set of states, 2. s_0 is the initial state, 3. S_F is the set of final states, 4. Σ is a set of actions, 5. $\text{Lab}_{\mathcal{T}} : S \rightarrow 2^{AP}$ is a labeling function, 6. $(s, \alpha, s') \in E \subseteq S \times (\Sigma \cup \mathbb{R}_{\geq 0}) \times S$ is a labeled transition relation with two kinds of transitions: discrete (if $\alpha \in \Sigma$) and delay (if $\alpha \in \mathbb{R}_{\geq 0}$) transitions.

Definition 2 (Semantics of a TA). Let $\mathcal{A} = (\mathbb{L}, \ell_0, \mathbb{C}, \Sigma, \mathbb{T}, \text{Inv}, \text{Lab}, \mathbb{F})$ be a TA. The semantics of $\mathcal{A}$ is given by the TTS $\mathcal{T}(\mathcal{A}) = (S, s_0, S_F, \Sigma, \text{Lab}_{\mathcal{T}}, E)$ where: 1. $S \subseteq \mathbb{L} \times \mathbb{R}_{\geq 0}^{\mathbb{C}}$, 2. $s_0 = (\ell_0, \mathbf{0})$ with $\mathbf{0} \models \text{Inv}(\ell_0)$, 3. $S_F \subseteq \mathbb{F} \times \mathbb{R}_{\geq 0}^{\mathbb{C}}$, 4. $\text{Lab}_{\mathcal{T}}((\ell, \nu)) = \text{Lab}(\ell) \cup \{g \in \mathbb{G}(\mathbb{C}) \mid \nu \models g\}$, 5. $E \subseteq S \times (\Sigma \cup \mathbb{R}_{\geq 0}) \times S$ is a transition relation defined by the following two rules:

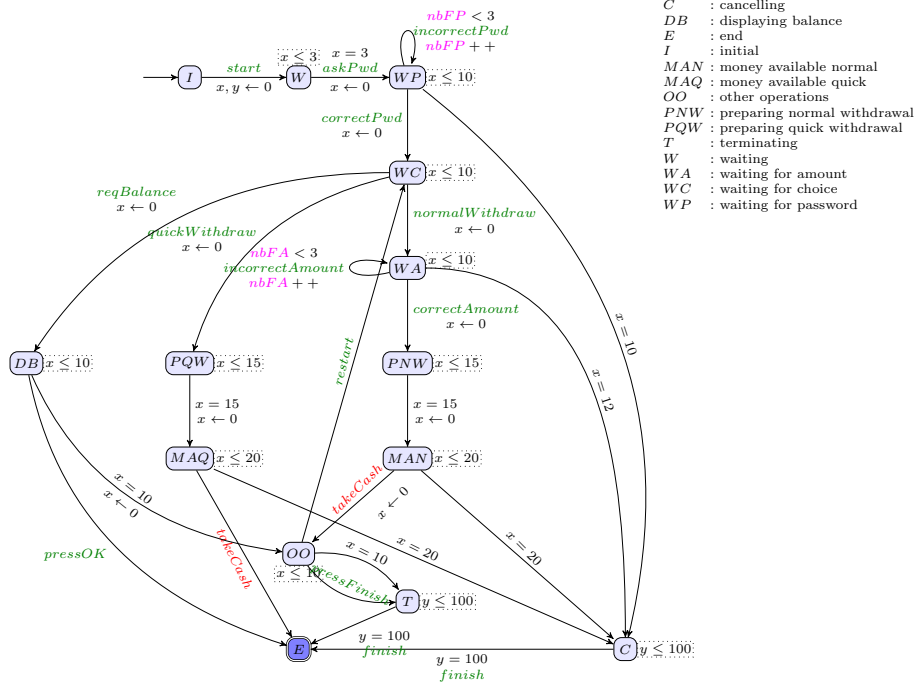


Fig. 1. Running example: ATM benchmark [3]

- **Discrete transition:** $(\ell_1, \nu) \xrightarrow{a} (\ell_2, \nu')$ for $a \in \Sigma$ iff $\ell_1 \xrightarrow{a, g, R} \ell_2, \nu \models g \wedge \nu' = [\nu]_R$ and $\nu' \models \text{Inv}(\ell_2)$ and,
- **Delay transition:** $(\ell_1, \nu) \xrightarrow{d} (\ell_1, \nu + d)$, for some $d \in \mathbb{R}_{\geq 0}$ iff $\nu + d \models \text{Inv}(\ell_1)$.

In the remaining of the paper, we consider a TA $\mathcal{A}$ and its associated TTS $\mathcal{T}$. A finite run of $\mathcal{A}$ is a finite path of $\mathcal{T}$ in which delay and discrete transitions alternate: $\rho = s_0 \xrightarrow{d_0} s'_1 \xrightarrow{a_0} s_1 \xrightarrow{d_1} s'_2 \xrightarrow{a_1} s_2 \dots s_{n-1} \xrightarrow{d_{n-1}} s'_n \xrightarrow{a_n} s_n$ or more compactly $\rho = s_0 \xrightarrow{d_0, a_0} s_1 \xrightarrow{d_1, a_1} s_2 \xrightarrow{d_2, a_2} s_3 \dots s_{n-1} \xrightarrow{d_{n-1}, a_{n-1}} s_n$. The last state s_n of a run ρ is denoted by $\text{last}(\rho)$. The *duration* of a run is the sum of the delays along the run: $\text{dur}(\rho) = \sum_{i=0}^{n-1} d_i$. Two runs are *duration-equivalent* if they have the same duration: we write $\rho \sim^t \rho'$ iff $\text{dur}(\rho) = \text{dur}(\rho')$. We denote by $\text{Paths}_{\mathcal{T}}(s)^+$ the set of all paths of $\mathcal{T}$ that start from s and end in a final state.

Example 1. As a running example and case study for our model and logic, we consider a TA model of an ATM system (Fig. 1). WP is a location, where we can stay as long as clock $x \leq 10$. Before $x > 10$, we must take one of the two outgoing transitions. If $x = 10$ we can go to the C location. If we take the *correctPwd* action, we go to the WC location, while resetting clock x . Note that this model

uses a discrete variable nbFP to count password failures; since it ranges over a finite domain, it is a syntactic sugar (and this model is still a TA).

2.2 Zone Graph and Predecessor Operator

Since $\mathcal{T}$ has infinitely many states due to the continuous nature of clocks, its semantics cannot be represented directly by a finite-state transition system. To obtain a finite symbolic abstraction, we use the *Zone Graph* (ZG) abstraction [22,10] (formally defined in Definition 3), which is the standard symbolic representation underlying most verification procedures TAs. Given a set of clocks $\mathbb{C}$, a *zone* is a set of clock valuations satisfying a conjunction of constraints of the form $x \bowtie d$ or $x - y \bowtie d$, where $x, y \in \mathbb{C}$, $d \in \mathbb{Z}$, and $\bowtie \in \{<, \leq, =, \geq, >\}$. Given a zone Z and a delay $d \in \mathbb{R}_{\geq 0}$, we denote by $Z^{\uparrow d}$ the zone $\{\nu + d \mid \nu \in Z\}$, which represents the evolution of clock valuations after a delay of d time units and $Z^{\uparrow} = \bigcup_{d \in \mathbb{R}_{\geq 0}} Z^{\uparrow d}$. A *symbolic state* is a pair $\mathcal{Z} = (\ell, Z)$ where $\ell \in \mathbb{L}$ is a location and Z is a zone over $\mathbb{C}$. A symbolic state (ℓ, Z) represents the set of concrete states $\{(\ell, \nu) \mid \nu \in Z\}$.

Definition 3 (Zone Graph). *The ZG of $\mathcal{A}$ is a transition system $\mathcal{G} = (\mathcal{S}, s_0, (\Sigma \cup \{\epsilon\}), \text{Lab}_{\mathcal{Z}}, \rightarrow)$ where: 1. $\mathcal{S}$ is a finite set of symbolic states (ℓ, Z) such that every valuation in Z satisfies the invariant of ℓ , i.e. $\forall \nu \in Z, \nu \models \text{Inv}(\ell)$, 2. $s_0 = (\ell_0, \{\mathbf{0}\})$ is the initial symbolic state, 3. Σ is the set of action labels of $\mathcal{A}$, item $\epsilon \notin \Sigma$ is a special symbol used to denote delay transitions in the symbolic semantics, 4. $\text{Lab}_{\mathcal{Z}}((\ell, Z)) = \text{Lab}(\ell) \cup \{g \in \mathbb{G}(\mathbb{C}) \mid \forall \nu \in Z, \nu \models g\}$, 5. $\rightarrow \subseteq \mathcal{S} \times (\Sigma \cup \{\epsilon\}) \times \mathcal{S}$ is the transition relation defined by:*

- $(\ell, Z) \xrightarrow{a} (\ell', Z')$ if there exists a transition $\ell \xrightarrow{a, g, R} \ell'$ in $\mathcal{A}$ such that $Z' = [(Z \cap g)]_R \cap \text{Inv}(\ell')$,
- $(\ell, Z) \xrightarrow{\epsilon} (\ell', Z')$ if $Z' = Z^{\uparrow} \cap \text{Inv}(\ell')$.

Using the standard normalization and abstraction operations on zones, the symbolic state space of $\mathcal{G}(\mathcal{A})$ is finite [22,10]. Hence, the ZG provides a finite abstraction of the infinite timed transition system $\mathcal{T}(\mathcal{A})$ while preserving the semantic information relevant to symbolic verification. For the remainder of the paper, we work on the ZG $\mathcal{G}(\mathcal{A})$ of $\mathcal{A}$. Given a set of symbolic states $\mathcal{P} \subseteq \mathcal{S}$, the set $\text{Pre}(\mathcal{P})$ of predecessor states of $\mathcal{P}$ is defined as: $\text{Pre}(\mathcal{P}) = \left\{ \mathcal{Z}' \in \mathcal{S} \mid \exists \mathcal{Z} \in \mathcal{S} \wedge \exists \alpha \in (\Sigma \cup \{\epsilon\}) \text{ such that } \mathcal{Z}' \xrightarrow{\alpha} \mathcal{Z} \right\}$. The computation of the set of predecessor states $\text{Pre}(\mathcal{P})$ consists of a delay step ($\text{Pre}^c(\mathcal{P})$) and discrete step ($(\text{Pre}^d(\mathcal{P}))$) [22]. The set of reachable states by a backward delay step is denoted by $(\text{Pre}^c(\mathcal{P}))$. The set of reachable states by a backward discrete step is denoted by $(\text{Pre}^d(\mathcal{P}))$.

3 Execution-Time Opacity Logic

Contrary to [6,5], where the secret is defined as the visit of a particular control location, we allow here an arbitrary private property over complete runs. In the following, we assume a secret property defined by a predicate ψ_{priv} .

Definition 4 (Execution-Time Opacity).

1. $\mathcal{T}$ satisfies existential ET-opacity at s with respect to ψ_{priv} , denoted $\mathcal{T}, s \models \exists\text{-ETO}(\psi_{priv})$ if there exist two duration-equivalent runs from s that differ on the satisfaction of ψ_{priv} , i.e. $\exists \rho, \rho' \in \text{Paths}_{\mathcal{T}}(s)^+, \rho \sim^t \rho' \wedge (\mathcal{T}, \text{last}(\rho) \models \psi_{priv}) \wedge (\mathcal{T}, \text{last}(\rho') \models \neg \psi_{priv})$
2. The system $\mathcal{A}$ satisfies universal ET-opacity at s with respect to ψ_{priv} , written $\mathcal{T}, s \models \forall\text{-ETO}(\psi_{priv})$ if for all possible durations of runs from s , there exist two duration-equivalent runs that differ on the satisfaction of ψ_{priv} , i.e. $\forall \rho \in \text{Paths}_{\mathcal{T}}(s)^+, \exists \rho' \in \text{Paths}_{\mathcal{T}}(s)^+, \rho \sim^t \rho' \wedge (\mathcal{T}, \text{last}(\rho) \models \psi_{priv}) \wedge (\mathcal{T}, \text{last}(\rho') \models \neg \psi_{priv})$

3.1 Execution-Time Opacity Logic (ETOL)

In this section, we introduce the syntax and semantics of ETOL, which is designed to capture ET-opacity. Intuitively, a formula of the form $\exists\varphi$ expresses that there exists at least one execution duration performed by two executions, one satisfying φ and the other violating φ . In other words, there exists a duration. Dually, a formula of the form $\forall\varphi$ expresses that this indistinguishability holds for all possible execution times.

Definition 5. Let AP a set of atomic propositions, $\mathbb{C}$ the set of clocks of $\mathcal{A}$ and $\mathbb{J}$ a non-empty set of clocks of the formula, where $\mathbb{C} \cap \mathbb{J} = \emptyset$. Formulas of ETOL are defined by the following (minimal) grammar:

$$\varphi ::= \top \mid p \mid \neg\varphi \mid g \mid \varphi \wedge \varphi \mid j.\varphi \mid \exists(\varphi \text{ U } \varphi) \mid \exists(\varphi \text{ R } \varphi) \mid \forall(\varphi \text{ U } \varphi) \mid \forall(\varphi \text{ R } \varphi)$$

where $p \in AP$, $\varphi \in \text{ETOL}$, $j \in \mathbb{J}$ and $g \in \mathbb{G}(\mathbb{C} \cup \mathbb{J})$.

It is possible to compare formula clocks $\mathbb{J}$ and automaton clocks $\mathbb{C}$ by means of clock constraints of the form g , which may involve both clocks from the TA and clocks introduced by the formula. The Boolean connectives $\perp$, $\vee$, and $\rightarrow$ are defined as usual. A clock j occurring in a formula of the form $j.\varphi$ is called a *freeze identifier* and binds the formula clock j within φ . Intuitively, the formula $j.\varphi$ holds in a state s if φ holds in the extended state obtained by resetting the value of clock j to 0 in s . Freeze identifiers can be used in conjunction with temporal operators to express common real-time requirements, such as punctuality constraints, bounded response times, or deadlines. ETOL introduces two dedicated ET-opacity operators: the *existential ET-opacity operator* $\exists$ corresponding to $\exists$ -ET-opacity, and the *universal ET-opacity operator* $\forall$ corresponding to $\forall$ -ET-opacity. For each ET-opacity operator in $\{\exists, \forall\}$, derived temporal operators are defined similarly:

$$\begin{aligned} \exists\text{F}\varphi &:= \exists(\top \text{ U } \varphi) & \forall\text{F}\varphi &:= \forall(\top \text{ U } \varphi) \\ \exists\text{G}\varphi &:= \exists(\perp \text{ R } \varphi) & \forall\text{G}\varphi &:= \forall(\perp \text{ R } \varphi) \\ \exists(\varphi \text{ W } \psi) &:= \exists(\psi \text{ R } (\varphi \vee \psi)) & \forall(\varphi \text{ W } \psi) &:= \forall(\psi \text{ R } (\varphi \vee \psi)). \end{aligned}$$

The size $|\varphi|$ of a formula φ is defined as the number of logical and temporal connectives occurring in φ .

Example 2. The formula $\varphi = \exists(j.(\varphi_1 \wedge j \leq 7) \cup \varphi_2)$ intuitively states that for every execution satisfying $j.(\varphi_1 \wedge j \leq 7) \cup \varphi_2$ (i.e. where φ_1 is true until φ_2 becomes true before 7 time units), there exists another execution with exactly the same total execution time measured by clock j for which this property does not hold. Consequently, an observer who can only observe execution times cannot determine whether the property $j.(\varphi_1 \wedge j \leq 7) \cup \varphi_2$ was satisfied.

Example 3. $\varphi = j_1.((\varphi_1 \wedge j_1 \leq 6) \wedge j_2. \exists(\varphi_2 \cup (\varphi_3 \wedge j_2 \leq 4)))$, the outer clock j_1 is reset before evaluating the whole formula inside the outer parentheses, and its constraint $j_1 \leq 6$ applies to the outer conjunct containing φ_1 . The inner clock j_2 is reset only for the nested opacity formula, where the constraint $j_2 \leq 4$ is interpreted relative to this second, local reset point.

Example 4. $\varphi = j. \exists((\varphi_1 \wedge x \leq 2) \cup (\varphi_2 \wedge j \leq 5 \wedge x \geq 1))$, the automaton clock x and the formula clock j play different roles. The constraint on x refers to the timing behaviour of the system itself, whereas the constraint on j measures time from the local reset introduced by the formula.

Unlike CTL, but for the same reasons as TCTL, ETOL does not include a next-time operator. Indeed, in our context, the next operator would not be meaningful since time is continuous and there is no unique next moment in time.

Definition 6 (Path semantics of $\cup$ (until) and R (release)). Let ρ be a finite run in $\text{Paths}_{\mathcal{T}}(s)^+$. We write :

- $\rho \models (\varphi_1 \cup \varphi_2)$, iff there exists $i \in \{0, \dots, n\}$ such that $\mathcal{T}, s_i \models \varphi_2$ and $\forall j < i, \mathcal{T}, s_j \models \varphi_1$,
- $\rho \models (\varphi_1 \mathsf{R} \varphi_2)$, iff either $\forall i \in \{0, \dots, n\}, \mathcal{T}, s_i \models \varphi_2$, or there exists $i \in \{0, \dots, n\}$ such that $\mathcal{T}, s_i \models \varphi_1 \wedge \varphi_2$ and $\forall j < i, \mathcal{T}, s_j \models \varphi_2$.

Formulas of ETOL are interpreted over TTS. We are now ready to formally define the semantics of ETOL formulas. An extended state s_μ over S is a triple (ℓ, ν, μ) , where $s = (\ell, \nu) \in S$ (in $\mathcal{T}$) and μ a valuation for the formula clocks in $\mathbb{J}$. To facilitate reading, from this point onward we will use only the symbol s for an extended state.

Definition 7 (ETOL semantics). Let $\mathbb{C}$ be the set of clocks of $\mathcal{A}$ and $\mathbb{J}$ a non-empty set of clocks of the formula, $p \in AP$, $g \in \mathbb{G}(\mathbb{C} \cup \mathbb{J})$. The satisfaction relation between $\mathcal{T}$, $\{\varphi, \psi\} \subseteq \text{ETOL}$ and an extended state $s = (\ell, \nu, \mu)$, denoted $\mathcal{T}, s \models \varphi$, is defined inductively as follows:

1. $\mathcal{T}, s \models \top$ for all state s ,
2. $\mathcal{T}, s \models p$ iff $p \in \text{Lab}_{\mathcal{T}}(s)$,
3. $\mathcal{T}, s \models \neg \varphi$ iff not $\mathcal{T}, s \models \varphi$ (notation $\mathcal{T}, s \not\models \varphi$),
4. $\mathcal{T}, s \models \varphi_1 \wedge \varphi_2$ iff $\mathcal{T}, s \models \varphi_1$ and $\mathcal{T}, s \models \varphi_2$,
5. $\mathcal{T}, s \models g$ iff $(\nu \cup \mu) \models g$,
6. $\mathcal{T}, s \models j.\varphi$ iff $\mathcal{T}, (l, \nu, [\mu]_{\{j\}}) \models \varphi$,
7. $\mathcal{T}, s \models \exists(\varphi_1 \cup \varphi_2)$ iff there exist runs $\rho, \rho' \in \text{Paths}_{\mathcal{T}}(s)^+$ such that $\text{dur}(\rho) = \text{dur}(\rho') \wedge \rho \models (\varphi_1 \cup \varphi_2) \wedge \rho' \not\models (\varphi_1 \cup \varphi_2)$,

8. $\mathcal{T}, s \models \exists(\varphi_1 \text{ R } \varphi_2)$ iff there exist runs $\rho, \rho' \in \text{Paths}_{\mathcal{T}}(s)^+$ such that $\text{dur}(\rho) = \text{dur}(\rho') \wedge \rho \models (\varphi_1 \text{ R } \varphi_2) \wedge \rho' \not\models (\varphi_1 \text{ R } \varphi_2)$,
9. $\mathcal{T}, s \models \forall(\varphi_1 \text{ U } \varphi_2)$ iff for every run $\rho \in \text{Paths}_{\mathcal{T}}(s)^+$, there exists a run $\rho' \in \text{Paths}_{\mathcal{T}}(s)^+$ such that $\text{dur}(\rho) = \text{dur}(\rho') \wedge (\rho \models (\varphi_1 \text{ U } \varphi_2) \Leftrightarrow \rho' \models (\varphi_1 \text{ U } \varphi_2))$,
10. $\mathcal{T}, s \models \forall(\varphi_1 \text{ R } \varphi_2)$ iff for every run $\rho \in \text{Paths}_{\mathcal{T}}(s)^+$, there exists a run $\rho' \in \text{Paths}_{\mathcal{T}}(s)^+$ such that $\text{dur}(\rho) = \text{dur}(\rho') \wedge (\rho \models (\varphi_1 \text{ R } \varphi_2) \Leftrightarrow \rho' \models (\varphi_1 \text{ R } \varphi_2))$

A formula φ is true in $\mathcal{T}$ iff $\mathcal{T}, s_0 \models \varphi$. Two formulas φ and ψ are equivalent (denoted by $\varphi \equiv \psi$) iff for any model $\mathcal{M}$ and state s of $\mathcal{M}$, $\mathcal{M}, s \models \varphi$ iff $\mathcal{M}, s \models \psi$. Let φ be any formula, then $\mathbf{Sat}(\varphi)$ denotes the set of states of $\mathcal{T}$ verifying φ , i.e. $\mathbf{Sat}(\varphi) = \{s \in S \mid \mathcal{T}, s \models \varphi\}$. The relationship between a TA and its semantics is defined as follows:

Definition 8. Let $\mathcal{A}$ be a TA and $\varphi \in \text{ETOL}$, then $\mathcal{A} \models \varphi$ iff $\mathcal{T} \models \varphi$.

4 Model Checking

In this section, we present a model-checking algorithm for ETOL and prove that its model-checking problem is PSPACE. The algorithm is based on a symbolic exploration of the timed state space through the zone-graph abstraction, combined with fixed-point computations for the branching-time temporal operators of ETOL. Its purpose is to decide, for a given TTS $\mathcal{T}$ and an ETOL formula φ , whether $\mathcal{T} \models \varphi$. For the remainder of this section, we fix an ETOL formula φ .

4.1 Adding a special clock

Before delving into the details of the algorithm, we introduce a new clock x_0 into our model. This clock plays a crucial role in ensuring ET-opacity by keeping track of the total duration of paths: by using x_0 as a global duration counter, we can maintain consistency across different execution paths and verify opacity properties effectively, as it allows comparison between the durations of secret and non-secret executions. This clock is added to the TA: $\mathcal{A} \oplus x_0 = (\mathbb{L}, \ell_0, \mathbb{C} \cup \{x_0\}, \Sigma, \mathbb{T}, \text{Inv}, \mathbb{F})$ and transposed to the TTS semantics $\mathcal{T} \oplus x_0$. Note that this new clock is never used in guards or invariants of $\mathcal{A}$, therefore the semantics is not affected by the addition of x_0 , except for the addition of this clock to valuations. In the following, we consider the TTS $\mathcal{T}$ and the ZG $\mathcal{G}$ of $\mathcal{A} \oplus x_0$ without changing the notation with this new clock.

4.2 Algorithm overview

For the intuition, the ETOL model-checking algorithm computes the set $\mathbf{Sat}(\varphi)$, which contains all states satisfying formula φ , followed by a check to determine whether the initial state belongs to this set. To check ETOL formulas, we use the ZG $\mathcal{G}$ abstraction of the semantics, which encodes the same properties of $\mathcal{T}$ in a

finite formalism. Then, the algorithm recursively computes $\mathbf{Sat}(\psi)$ for all subformulas ψ of φ . The core of the algorithm consists of a fixed-point computation that iterates over the ZG until stability is reached (i.e. the iteration converges and no new zones nor states are added). We therefore ensure a complexity that is polynomial in the size of the ZG and exponential in the size of the formula, leading to an overall PSPACE complexity for the model checking problem of ETOL (Theorem 1). Recall that, to ensure the finiteness and tractability, we apply standard zone-extrapolation abstractions (e.g. see [12]), which over-approximate and merge clock zones beyond the relevant bounds.

Algorithm 1 ETOL Model Checking

Require: A TA $\mathcal{A}$ and φ is an ETOL formula.

Ensure: $\mathbf{Sat}(\psi) \leftarrow \left\{ s \in \mathcal{S} \mid \mathcal{G}, s \models \varphi \right\}$.

```

1: Compute  $\mathcal{G}(\mathcal{A} \oplus x_0, \varphi)$ 
2: for all  $i \leq |\varphi|$  do
3:   for all  $\psi \in \mathbf{Sub}(\varphi)$  with  $|\psi| = i$  do
4:     switch ( $\psi$ ) do
5:       case  $\psi = \top$ 
6:          $\mathbf{Sat}(\psi) \leftarrow \mathcal{S}$ 
7:       case  $\psi = p$ 
8:          $\mathbf{Sat}(\psi) \leftarrow \left\{ s \in \mathcal{S} \mid p \in \text{Lab}_{\mathcal{Z}}(s) \right\}$ 
9:       case  $\psi = \neg\psi_1$ 
10:         $\mathbf{Sat}(\psi) \leftarrow \mathcal{S} \setminus \mathbf{Sat}(\psi_1)$ 
11:      case  $\psi = g$ 
12:         $\mathbf{Sat}(\psi) \leftarrow \left\{ s \in \mathcal{S} \mid \forall \nu \in Z, \nu \models g \right\}$ 
13:      case  $\psi = \psi_1 \wedge \psi_2$ 
14:         $\mathbf{Sat}(\psi) \leftarrow \mathbf{Sat}(\psi_1) \cap \mathbf{Sat}(\psi_2)$ 
15:      case  $\psi = j.\psi_1$ 
16:         $\mathbf{Sat}(\psi) \leftarrow \left\{ s \in \mathcal{S} \mid [\mu]_{\{j\}}(s) \in \mathbf{Sat}(\psi_1) \right\}$ 
17:      case  $\psi = \exists(\psi_1 \mathbf{U} \psi_2)$ 
18:         $\mathbf{Sat}(\psi) \leftarrow \mathbf{OpU}_{\exists}(\mathbf{Sat}(\psi_1), \mathbf{Sat}(\psi_2))$ 
19:      case  $\psi = \forall(\psi_1 \mathbf{U} \psi_2)$ 
20:         $\mathbf{Sat}(\psi) \leftarrow \mathbf{OpU}_{\forall}(\mathbf{Sat}(\psi_1), \mathbf{Sat}(\psi_2))$ 
21:      case  $\psi = \exists(\psi_1 \mathbf{R} \psi_2)$ 
22:         $\mathbf{Sat}(\psi) \leftarrow \mathbf{OpR}_{\exists}(\mathbf{Sat}(\psi_1), \mathbf{Sat}(\psi_2))$ 
23:      case  $\psi = \forall(\psi_1 \mathbf{R} \psi_2)$ 
24:         $\mathbf{Sat}(\psi) \leftarrow \mathbf{OpR}_{\forall}(\mathbf{Sat}(\psi_1), \mathbf{Sat}(\psi_2))$ 

```

4.3 Algorithm details

Algorithm 1 takes as input the ZG with an ETOL formula φ , and constructs the zone graph $\mathcal{G}(\mathcal{A} \oplus x_0, \varphi)$, and returns the set of states of that satisfy φ . **Algorithm 1** proceeds in a standard bottom-up fashion: for every subformula ψ of φ , it computes the satisfaction set $\mathbf{Sat}(\psi) = \{s \in \mathcal{S} \mid \mathcal{G}, s \models \psi\}$. The computation of $\mathbf{Sat}(\psi)$ is immediate when ψ is $\top$, an atomic proposition p , or a clock constraint g . Negation and conjunction are handled by set difference and intersection, respectively. The non-trivial cases correspond to the opacity-temporal formulas $(\psi_1 \mathbf{U} \psi_2)$ and $(\psi_1 \mathbf{R} \psi_2)$, evaluated under the existential and universal opacity operators $\ominus$ and $\forall$. To deal with these cases, the algorithm relies on two procedures $\mathbf{OpU}_{\ominus}$ and $\mathbf{OpU}_{\forall}$ that compute the satisfaction sets for the $\mathbf{U}$ operator under the existential and universal ET-opacity operators, respectively. The same idea applies to the $\mathbf{R}$ operator, with procedures $\mathbf{OpR}_{\ominus}$ and $\mathbf{OpR}_{\forall}$. These procedures are defined in [Section 4.5](#) and are based on fixed-point computations over the ZG.

4.4 Execution-time predecessor operators

To evaluate the execution-time opacity modalities, we introduce two symbolic predecessor operators over the zone graph. More precisely, for a set of symbolic states $\mathbb{X} \subseteq \mathcal{S}$:

- the operator $Pre_{\ominus}(\mathbb{X})$ returns the set of symbolic predecessor states satisfying the existential execution-time opacity condition;
- the operator $Pre_{\forall}(\mathbb{X})$ returns the set of symbolic predecessor states satisfying the universal execution-time opacity condition.

Intuitively, a symbolic state should belong to such a predecessor set if, from that state, the system admits two alternative continuations: one reaching a state in $\mathbb{X}$ and another reaching a state in $\mathcal{S} \setminus \mathbb{X}$, while the corresponding observable durations cannot be distinguished by an external observer. For the existential operator, it is sufficient that there exists at least one common execution-time value compatible with both alternatives. For the universal operator, every execution-time value leading to one side must also be compatible with the other side, and conversely. To formalize this condition, we use the global clock x_0 , whose value records the total elapsed execution time. We therefore associate with each symbolic state the set of execution-time values represented by its zone, and define the predecessor operators by comparing these sets symbolically. These operators are used in the fixed-point procedures $\mathbf{OpU}_{\ominus}$, $\mathbf{OpU}_{\forall}$, $\mathbf{OpR}_{\ominus}$, and $\mathbf{OpR}_{\forall}$ presented in [Section 4.5](#).

Definition 9. Let $\mathbb{X}$ be a set of symbolic states, we write:

$$Pre_{\ominus}(\mathbb{X}) = \left\{ Z \mid \left\{ \begin{array}{l} Z \in (Pre(\mathbb{X}) \cap Pre(\mathcal{S} \setminus \mathbb{X})) \wedge \\ \exists Z' \in Pre(\mathbb{X}), \exists Z'' \in Pre(\mathcal{S} \setminus \mathbb{X}) \\ \exists \nu \in Z, \exists \nu' \in Z', \exists \nu'' \in Z'' \text{ such that} \\ \nu(x_0) = \nu'(x_0) = \nu''(x_0) \end{array} \right\} \right\}$$

Algorithm 2 Backward search for computing $\mathbf{OpU}_{\ominus}$

Require: An ETOL formula $\ominus(\psi_1 \cup \psi_2)$.

Ensure: $\mathbf{Sat}(\psi_1 \cup \psi_2) \leftarrow \left\{ s \in S \mid \mathcal{M}, s \models \ominus(\psi_1 \cup \psi_2) \right\}$.

- 1: $X \leftarrow \emptyset$
 - 2: $Y \leftarrow \mathbf{Sat}(\psi_2)$
 - 3: **while** $Y \neq X$ **do**
 - 4: $X \leftarrow Y$
 - 5: $Y \leftarrow \psi_2 \cup (\psi_1 \cap \text{Pre}_{\ominus}(X))$
 - 6: **return** Y
-

Definition 10. Let $\mathbb{X}$ be a set of symbolic states, we write:

$$\text{Pre}_{\oplus}(\mathbb{X}) = \left\{ Z \mid \left\{ \begin{array}{l} Z \in \text{Pre}(\mathbb{X}) \cap \text{Pre}(\mathcal{S} \setminus \mathbb{X}) \wedge \\ \forall Z' \in \text{Pre}(\mathbb{X}), \exists Z'' \in \text{Pre}(\mathcal{S} \setminus \mathbb{X}) \\ \exists \nu \in Z, \exists \nu' \in Z', \exists \nu'' \in Z'' \text{ such that} \\ \nu(x_0) = \nu'(x_0) = \nu''(x_0) \wedge \\ \forall Z'' \in \text{Pre}(\mathcal{S} \setminus \mathbb{X}), \exists Z' \in \text{Pre}(\mathbb{X}) \\ \exists \nu \in Z, \exists \nu' \in Z', \exists \nu'' \in Z'' \text{ such that} \\ \nu(x_0) = \nu'(x_0) = \nu''(x_0) \end{array} \right\} \right\}$$

Proposition 1. Let $X \subseteq \mathcal{S}$. Then $\text{Pre}_{\ominus}(X) \subseteq \text{Pre}(\mathcal{P}) \cap \text{Pre}(\mathcal{S} \setminus X)$. Moreover, for every $Z \in \text{Pre}_{\ominus}(X)$, there exist symbolic states $Z' \in \text{Pre}(X)$ and $Z'' \in \text{Pre}(\mathcal{S} \setminus X)$ and valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ such that $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$.

Proposition 2. Let $X \subseteq \mathcal{S}$. Then $\text{Pre}_{\oplus}(X) \subseteq \text{Pre}(X) \cap \text{Pre}(\mathcal{S} \setminus X)$. Moreover, for every $Z \in \text{Pre}_{\oplus}(X)$, the following two conditions hold:

1. for every $Z' \in \text{Pre}(X)$, there exists $Z'' \in \text{Pre}(\mathcal{S} \setminus X)$ and valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ such that $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$,
2. for every $Z'' \in \text{Pre}(\mathcal{S} \setminus X)$, there exists $Z' \in \text{Pre}(X)$ and valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ such that $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$.

4.5 Procedures

As in the standard zone-based model checking for branching-time timed logics, the ET-opacity operators are evaluated by fixed-point computations over symbolic predecessor sets:

- For the $\mathbf{U}$ operator, the fixed-point iteration starts from $\mathbf{Sat}(\psi_2)$ and repeatedly adds symbolic states that satisfy ψ_1 and can reach the current set via the appropriate predecessor operator. We present $\mathbf{OpU}_{\ominus}$ in [Algorithm 2](#) and $\mathbf{OpU}_{\oplus}$ in [Algorithm 4](#).
- The $\mathbf{R}$ operator is handled in a dual and analogous way, using a fixed-point computation that also relies on the corresponding predecessor operator. $\mathbf{OpR}_{\ominus}$ and $\mathbf{OpR}_{\oplus}$ are presented in [Algorithms 3](#) and [5](#), respectively.

Algorithm 3 Backward search for computing $\text{OpR}_{\ominus}$

Require: An ETOL formula $\ominus(\psi_1 \text{ R } \psi_2)$.

Ensure: $\text{Sat}(\psi_1 \text{ R } \psi_2) \leftarrow \{s \in S \mid \mathcal{M}, s \models \ominus(\psi_1 \text{ R } \psi_2)\}$.

```
1:  $X \leftarrow \emptyset$ 
2:  $Y \leftarrow \text{Sat}(\psi_2)$ 
3: while  $Y \neq X$  do
4:    $X \leftarrow Y$ 
5:    $Y \leftarrow \psi_2 \cap (\psi_1 \cup \text{Pre}_{\ominus}(X))$ 
6: return  $Y$ 
```

Algorithm 4 Backward search for computing *ProcedureOU* for all

Require: An ETOL subformula $\forall(\psi_1 \cup \psi_2)$.

Ensure: $\text{Sat}(\forall(\psi_1 \cup \psi_2))$.

```
1:  $X \leftarrow \emptyset$ 
2:  $Y \leftarrow \text{Sat}(\psi_2)$ 
3: while  $Y \neq X$  do
4:    $X \leftarrow Y$ 
5:    $Y \leftarrow \text{Sat}(\psi_2) \cup (\text{Sat}(\psi_1) \cap \text{Pre}_{\forall}(X))$ 
6: return  $Y$ 
```

4.6 Termination, soundness and complexity

Termination of [Algorithm 1](#) follows from the finiteness of the ZG.

Proposition 3 (Termination). *Let $\mathcal{A}$ be a TA and φ be an ETOL formula. [Algorithm 1](#) always terminates on input $\mathcal{G}(\mathcal{A} \oplus x_0, \varphi)$.*

Proposition 4 (Soundness and Completeness). *Let φ be an ETOL formula. Then [Algorithm 1](#) is sound and complete: for every subformula $\psi \in \text{Sub}(\varphi)$, $\text{Sat}(\psi) = \{s \in S \mid \mathcal{G}, s \models \psi\}$. In particular, for every initial symbolic state s_0 , $\mathcal{G}, s_0 \models \varphi$ iff $s_0 \in \text{Sat}(\varphi)$.*

Algorithm 5 Backward search for computing $\text{OpR}_{\forall}$

Require: An ETOL subformula $\forall(\psi_1 \text{ R } \psi_2)$.

Ensure: $\text{Sat}(\forall(\psi_1 \text{ R } \psi_2))$.

```
1:  $X \leftarrow \emptyset$ 
2:  $Y \leftarrow \text{Sat}(\psi_2)$ 
3: while  $Y \neq X$  do
4:    $X \leftarrow Y$ 
5:    $Y \leftarrow \text{Sat}(\psi_2) \cap (\text{Sat}(\psi_1) \cup \text{Pre}_{\forall}(X))$ 
6: return  $Y$ 
```

Finally, the following theorem establishes the complexity of our model checking algorithm.

Theorem 1. *The model-checking problem for ETOL over TA belongs to PSPACE.*

5 Case study and Implementation

5.1 Case study

Consider the ATM system depicted in Fig. 1, which models a typical user session with an ATM machine. The system includes various states representing different stages of the interaction, such as starting the session, entering a password, choosing an operation, and taking cash. The transitions between states are labeled with actions (e.g. *start*, *askPwd*, *correctPwd*) and are associated with timing constraints that reflect the expected durations of user interactions and system responses. We assume a timing attacker who observes only the *total execution time*, i.e. the elapsed time from the session start to the final state. The secret is whether cash is actually dispensed, modeled by the action *takeCash*. The model relies on two clocks: a local clock x for user ATM interaction steps (inputs and responses), and a global clock y that bounds the overall session duration (invariants $y \leq 100$ in the terminating and cancelling phases). A session starts with *start*, which resets the clocks, and displays a welcome screen for 3 time units before requesting a password (*askPwd*). If the user enters an incorrect password (*incorrectPwd*), the failure counter is incremented; after three failures, or if no response is given within 10 time units, the session is cancelled. After a correct password (*correctPwd*), the user chooses among quick withdrawal, normal withdrawal, or balance inquiry. Quick and normal withdrawals involve a preparation phase lasting 15 time units before the cash becomes available; if cash is not taken within 20 time units, the session is canceled. In the normal withdrawal branch, the user must additionally enter an amount; three invalid attempts or a timeout of 12 time units leads to cancellation. A balance inquiry displays the balance and then allows either termination or a return to the choice state. Finally, after a successful cash withdrawal (or other operations), the user may restart or finish; if no response is given within 10 time units the session proceeds to termination. This model provides a realistic setting for analyzing ET-opacity: although the global bound $y \leq 100$ limits the maximum observable duration, timing differences between interaction patterns (e.g. quick vs. normal withdrawal, repeated failures, or early termination) may still leak information unless opacity constraints are enforced. We can express the following security properties:

- **Existential opacity of successful cash withdrawal.** There exists at least one observable execution time for which an external observer cannot determine, from total duration alone, whether the execution eventually performs the cash-withdrawal action: $\varphi_1 := \exists(\top \cup \textit{takeCash})$.

- **Universal bounded opacity of money availability.** An observer should not be able to infer, from the total execution time, whether the system reaches one of the states where money is available, within the global session horizon: $\varphi_2 := j. \bigvee (\neg(MAQ \vee MAN) \cup (E \wedge j \leq 100))$.
- **Local bounded opacity after entering a critical phase.** There exists an execution time for which, once a relevant phase is entered, taking cash within a local delay remains indistinguishable: $\varphi_3 := \bigoplus (\top \cup j. (\text{takeCash} \wedge j \leq 20))$.
- **Opacity combining automaton clocks and formula clocks.** The preparation phase of a quick withdrawal and the final withdrawal action should remain ambiguous even when both system time and specification time are taken into account: $\varphi_4 := \bigvee ((PQW \wedge x \leq 15) \cup j. (\text{takeCash} \wedge j \leq 5))$.
- **Nested freeze clocks with global and local scopes.** The logic can also express opacity properties involving several temporal reference points: $\varphi_5 := j_1. ((\neg C \wedge j_1 \leq 100) \wedge j_2. \bigoplus (\text{correctPwd} \cup (\text{takeCash} \wedge j_2 \leq 30)))$.
- **Universal opacity of fixed-time termination.** The end of the ATM session should remain ambiguous even when it occurs at a precise local delay: $\varphi_6 := \bigvee (\neg \text{takeCash} \cup j. (E \wedge j = 10))$.

5.2 Implementation

Here, we present the implementation⁵ of the model checking algorithm and its integration with the VITAMIN tool [20] as a verification component to handle the verification of ETOL formulas. The verification tool relies on zone-based symbolic exploration using *Difference Bound Matrices* (DBMs) [11] [18], and supports freeze clocks, opacity operators $\bigoplus$ and $\bigvee$ as defined in Definition 7. Our implementation is written in Python 3.11. The tool takes a file representing a model and a formula as input and applies the corresponding model-checking algorithm for the ETOL logic. TAs are represented as graphs and implemented via adjacency matrices as specified in the input file. We evaluated our ETOL model checker on a machine with the following specifications: Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz (4 cores / 8 threads), 16GB DDR4 RAM.

5.3 Experiments

Proof of concept, Scalability We ran the checker on the ATM timed automaton of Fig. 1 and verified the six properties φ_1 – φ_6 defined in Section 5.1. For each property, the tool reports whether the initial state satisfies the formula, and when applicable it outputs a witness execution together with an alternative execution of identical total duration. The tool completed the verification within 0.0285 seconds. To assess scalability, we conducted additional experiments on automatically generated TA, parameterized by the number of locations, the number of transitions, the number of clocks, and maximal constants appearing in guards/invariants. We generate multiple random instances and measure the memory used for each property. We report the average runtime over all generated instances, as well as standard deviation. In our synthetic ATM experiments, the

⁵ Source code available at: <https://github.com/jortizve/ETOL>

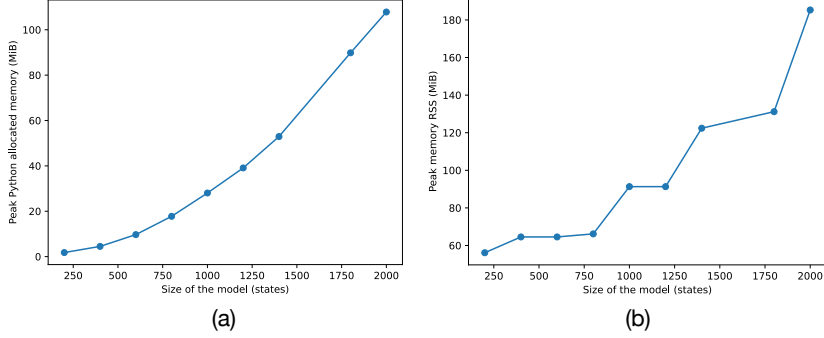


Fig. 2. Average Python memory allocation (MB).

analyzed models are not the handcrafted ATM instance of the case study: they are generated automatically by extending the ATM core with additional session modules (e.g. authentication, operation selection, and withdrawal/balance branches). This construction scales the system while preserving the semantics of the key control states and atomic propositions used in the opacity specifications. Model size is measured by the total number of states and transitions, atomic propositions on the added states are assigned automatically, while the original ATM labeling is kept unchanged, yielding diverse yet semantically consistent instances. To complement runtime measurements, we report peak memory consumption during model checking using two indicators: (i) the peak Resident Set Size (RSS), which captures the overall memory footprint of the process (including Python objects and native allocations), and (ii) the peak Python-allocated memory measured by `tracemalloc`, which reflects memory allocated by Python-level objects only. As shown in Figure 2, both measures increase smoothly with the number of states, confirming a controlled growth of the explored symbolic state space. The RSS curve is consistently higher than the `tracemalloc` curve, as expected, since RSS also includes interpreter overhead, libraries, and non-Python memory.

6 Conclusion

In this work, we studied the verification of ET-opacity in real-time systems modeled by TAs. Since timed opacity is undecidable for TAs in general [17], we focused on the decidable execution-time observation setting. We extended the location-based notion of ET-opacity from [6] to a more general framework where secrets are specified by ETOL formulas. We introduced ETOL, together with a symbolic model-checking procedure based on zone exploration and dedicated execution-time predecessor operators. We prove the correctness of our algorithm and establish its PSPACE complexity. We also implemented the approach and evaluated it on an ATM case study, illustrating the practical verification of execution-time opacity properties. As future work, we plan to enrich ETOL with quantitative, epistemic, strategic, and probabilistic features.

References

- [1] Alur, R., Dill, D.: A theory of timed automata. *Theoretical computer science* **126**(2), 183–235 (1994)
- [2] Ammar, I., Touati, Y.E., Yeddes, M., Mullins, J.: Bounded opacity for timed systems. *J. Inf. Secur. Appl.* **61**, 102926 (2021). <https://doi.org/10.1016/J.JISA.2021.102926>
- [3] André, É., Bolat, S., Lefaucheux, E., Marinho, D.: strategFTO: Untimed control for timed opacity. In: Artho, C., Ölveczky, P. (eds.) *FTSCS 2022*. pp. 27–33. *FTSCS 2022*, ACM, New York, NY, USA (2022). <https://doi.org/10.1145/3563822.3568013>
- [4] André, É., Dépernet, S., Lefaucheux, E.: The bright side of timed opacity. In: Ogata, K., Méry, D., Sun, M., Liu, S. (eds.) *ICFEM 2024*. pp. 51–69. *LNCS*, Springer, Hiroshima, Japan (2024). https://doi.org/10.1007/978-981-96-0617-7_4
- [5] André, E., Lefaucheux, E., Lime, D., Marinho, D., Sun, J.: Configuring timing parameters to ensure execution-time opacity in timed automata. In: ter Beek, M.H., Dubslaff, C. (eds.) *TiCSA@ETAPS 2023*. *EPTCS*, vol. 392, pp. 1–26 (2023). <https://doi.org/10.4204/EPTCS.392.1>
- [6] André, É., Lime, D., Marinho, D., Sun, J.: Guaranteeing timed opacity using parametric timed model checking. *ACM Trans. Softw. Eng. Methodol.* **31**(4) (Sep 2022). <https://doi.org/10.1145/3502851>
- [7] Behrmann, G., Bouyer, P., Larsen, K.G., Pelánek, R.: Lower and upper bounds in zone-based abstractions of timed automata. *International Journal on Software Tools for Technology Transfer* **8**(3), 204–215 (2006)
- [8] Behrmann, G., David, A., Larsen, K.G.: A tutorial on uppaal. In: Bernardo, M., Corradini, F. (eds.) *Formal Methods for the Design of Real-Time Systems*, International School on Formal Methods for the Design of Computer, Communication and Software Systems, SFM-RT 2004. *LNCS*, vol. 3185, pp. 200–236. Springer, Bertinoro, Italy (2004). https://doi.org/10.1007/978-3-540-30080-9_7
- [9] Benattar, G., Cassez, F., Lime, D., Roux, O.H.: Control and synthesis of non-interferent timed systems. *IJC* **88**(2), 217–236 (2015). <https://doi.org/10.1080/00207179.2014.944356>
- [10] Bengtsson, J., Yi, W.: Timed automata: Semantics, algorithms and tools. In: *Lectures on Concurrency and Petri Nets*, pp. 87–124. *LNCS* (2004). <https://doi.org/10.1007/b98282>
- [11] Berthomieu, B., Menasche, M.: An enumerative approach for analyzing time petri nets. In: *Proceedings of the IFIP Congress, Paris, France, 19–23 September 1983*. pp. 35–40 (1983)
- [12] Bouyer, P.: Forward analysis of updatable timed automata. *Formal Methods Syst. Des.* **24**(3), 281–320 (2004)
- [13] Brumley, D., Boneh, D.: Remote timing attacks are practical. *Computer Networks* **48**(5), 701–716 (2005). <https://doi.org/10.1016/J.COMNET.2005.01.010>
- [14] Bryans, J.W., Koutny, M., Ryan, P.Y.A.: Modelling opacity using petri nets. *Workshop on Security Issues in Concurrency* pp. 141–150 (2005)
- [15] Bryans, J.W., Koutny, M., Ryan, P.Y.A.: Opacity generalised to transition systems. *International Journal of Information Security* **7**, 421–435 (2008)
- [16] Bérard, B., Kouchnarenko, O., Mullins, J., Sassolas, M.: Probabilistic opacity in refinement-based modeling (2015). <https://doi.org/10.48550/arxiv.1510.04316>

- [17] Cassez, F.: The dark side of timed opacity. In: Park, J.H., Chen, H., Atiquzzaman, M., Lee, C., Kim, T., Yeo, S. (eds.) ISA 2009. LNCS, vol. 5576, pp. 21–30. Springer, Seoul, Korea (2009). https://doi.org/10.1007/978-3-642-02617-1_3
- [18] Cortes, D., Leneutre, J., Malvone, V., Ortiz, J.: A timed obstruction logic for dynamic game models (2025), <https://arxiv.org/abs/2510.06045>
- [19] Do, C.T., Tran, N.H., Hong, C., Kamhoua, C.A., Kwiat, K.A., Blasch, E., Ren, S., Pissinou, N., Iyengar, S.S.: Game theory for cyber security and privacy. ACM Comput. Surv. **50**(2) (May 2017), <https://doi.org/10.1145/3057268>
- [20] Ferrando, A., Malvone, V.: Vitamin: A compositional framework for model checking of multi-agent systems (2024). <https://doi.org/10.48550/arxiv.2403.02170>
- [21] Gerking, C., Schubert, D., Bodden, E.: Model checking the information flow security of real-time systems. In: Payer, M., Rashid, A., Such, J.M. (eds.) ESSoS 2018. LNCS, vol. 10953, pp. 27–43. Springer, Paris, France (2018). https://doi.org/10.1007/978-3-319-94496-8_3
- [22] Henzinger, T.A., Nicollin, X., Sifakis, J., Yovine, S.: Symbolic model checking for real-time systems. IC **111**(2), 193–244 (1994). <https://doi.org/10.1006/INCO.1994.1045>
- [23] Jérémie Dubreil, P.A.M., Marchand, H.: Opacity enforcing control synthesis. Automatica **46**, 605–613 (2010)
- [24] Kocher, P.C.: Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In: Koblitz, N. (ed.) CRYPTO 1996. LNCS, vol. 1109, pp. 104–113. Springer, Santa Barbara, CA, USA (1996). https://doi.org/10.1007/3-540-68697-5_9
- [25] Mazaré, L.: Using unification for opacity properties. Proceedings of the 4th IFIP WG1 **7**, 165–176 (2004)
- [26] Nielson, F., Nielson, H.R., Vasilikos, P.: Information flow for timed automata. In: Aceto, L., Bacci, G., Bacci, G., Ingólfssdóttir, A., Legay, A., Mardare, R. (eds.) Models, Algorithms, Logics and Tools. LNCS, vol. 10460, pp. 3–21. Springer (2017). https://doi.org/10.1007/978-3-319-63121-9_1
- [27] Saboori, A., Hadjicostis, C.G.: Verification of initial-state opacity in security applications of discrete event systems. Information Sciences **246**, 115–132 (2013)
- [28] Vasilikos, P., Nielson, F., Nielson, H.R.: Secure information release in timed automata. In: Bauer, L., Küsters, R. (eds.) POST 2018. LNCS, vol. 10804, pp. 28–52. Springer, Thessaloniki, Greece (2018). https://doi.org/10.1007/978-3-319-89722-6_2
- [29] Waga, M., André, É.: Hyper parametric timed CTL. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. **43**(11), 4286–4297 (2024). <https://doi.org/10.1109/TCAD.2024.3443704>
- [30] Wang, L., Zhan, N.: Decidability of the initial-state opacity of real-time automata. In: Symposium on Real-Time and Hybrid Systems. LNCS, Springer (2018). https://doi.org/10.1007/978-3-030-01461-2_3

Appendix

Proposition 1 *Let $X \subseteq \mathcal{S}$. Then $Pre_{\ominus}(X) \subseteq Pre(\mathcal{P}) \cap Pre(\mathcal{S} \setminus X)$. Moreover, for every $Z \in Pre_{\ominus}(X)$, there exist symbolic states $Z' \in Pre(X)$ and $Z'' \in Pre(\mathcal{S} \setminus X)$ and valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ such that $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$.*

Proof. Let $X \subseteq \mathcal{S}$, and let $Z \in Pre_{\ominus}(X)$. By definition of $Pre_{\ominus}(X)$, we have: $Z \in Pre(X) \cap Pre(\mathcal{S} \setminus X)$, and there exist symbolic states $Z' \in Pre(X)$ and $Z'' \in Pre(\mathcal{S} \setminus X)$ such that there exist valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ satisfying $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$. The first part immediately implies that $Z \in Pre(X) \cap Pre(\mathcal{S} \setminus X)$, and since Z was arbitrary, we obtain $Pre_{\ominus}(X) \subseteq Pre(X) \cap Pre(\mathcal{S} \setminus X)$. The second part is exactly the witness condition given by the definition of $Pre_{\ominus}(X)$: there exist symbolic predecessors Z' and Z'' , respectively associated with X and its complement, together with valuations in the corresponding zones, such that the value of the global clock x_0 coincides in the three valuations.

Proposition 2 *Let $X \subseteq \mathcal{S}$. Then $Pre_{\oplus}(X) \subseteq Pre(X) \cap Pre(\mathcal{S} \setminus X)$. Moreover, for every $Z \in Pre_{\oplus}(X)$, the following two conditions hold:*

1. *for every $Z' \in Pre(X)$, there exists $Z'' \in Pre(\mathcal{S} \setminus X)$ and valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ such that $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$,*
2. *for every $Z'' \in Pre(\mathcal{S} \setminus X)$, there exists $Z' \in Pre(X)$ and valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ such that $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$.*

Proof. Let $X \subseteq \mathcal{S}$, and let $Z \in Pre_{\oplus}(X)$. By definition of $Pre_{\oplus}(X)$, we first have $Z \in Pre(X) \cap Pre(\mathcal{S} \setminus X)$. Therefore, $Z \in Pre(X) \cap Pre(\mathcal{S} \setminus X)$, and, since Z was arbitrary, we obtain $Pre_{\oplus}(X) \subseteq Pre(X) \cap Pre(\mathcal{S} \setminus X)$. It remains to prove the two matching conditions. By the definition of $Pre_{\oplus}(X)$, for every symbolic state $Z' \in Pre(X)$, there exists a symbolic state $Z'' \in Pre(\mathcal{S} \setminus X)$ such that there exist valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ satisfying $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$. This proves Item (1). Similarly, again by the definition of $Pre_{\oplus}(X)$, for every symbolic state $Z'' \in Pre(\mathcal{S} \setminus X)$, there exists a symbolic state $Z' \in Pre(X)$ such that there exist valuations $\nu \in Z$, $\nu' \in Z'$, $\nu'' \in Z''$ satisfying $\nu(x_0) = \nu'(x_0) = \nu''(x_0)$. This proves Item (2). Hence both matching conditions hold, and the result follows.

Proposition 3 (Termination) *Let $\mathcal{A}$ be a TA and φ be an ETOL formula. Algorithm 1 always terminates on input $\mathcal{G}(\mathcal{A} \oplus x_0, \varphi)$.*

Proof. The zone graph $\mathcal{G}(\mathcal{A} \oplus x_0)$ is finite under the standard zone extrapolation used in the construction. Hence its set $\mathcal{S}$ of symbolic states is finite, and therefore the powerset $2^{\mathcal{S}}$ is finite as well. Moreover, the set of subformulas $\mathbf{Sub}(\varphi)$ is finite, since it is induced by the finite syntax of φ . Thus, Algorithm 1 performs only finitely many top-level labeling steps. For each subformula, the algorithm

computes a satisfaction set $\mathbf{Sat}(\psi) \subseteq \mathcal{S}$. By assumption, the fixed-point procedures used for the opacity-temporal cases do not revisit previously computed subsets of $\mathcal{S}$. Since there are only finitely many subsets of $\mathcal{S}$, each such procedure can perform only finitely many iterations before stabilizing or exhausting all possible candidates. Consequently, every call appearing in Algorithm 1 terminates after finitely many steps. Since the number of such calls is finite, Algorithm 1 terminates.

Proposition 4 (Soundness and Completeness) *Let φ be an ETOL formula. Then Algorithm 1 is sound and complete: for every subformula $\psi \in \mathbf{Sub}(\varphi)$, $\mathbf{Sat}(\psi) = \{s \in \mathcal{S} \mid \mathcal{G}, s \models \psi\}$. In particular, for every initial symbolic state s_0 , $\mathcal{G}, s_0 \models \varphi$ iff $s_0 \in \mathbf{Sat}(\varphi)$.*

Proof. We proceed by structural induction on the syntax of subformulas $\psi \in \mathbf{Sub}(\varphi)$. The base cases follow directly from the semantic clauses. If $\psi = \top$, then $\top$ is true in every symbolic state, and Algorithm 1 returns $\mathcal{S}$. Hence $\mathbf{Sat}(\top) = \mathcal{S} = \{s \in \mathcal{S} \mid \mathcal{G}, s \models \top\}$. If $\psi = p$, the algorithm returns precisely the symbolic states whose labels contain p . This is exactly the semantic clause for atomic propositions. If $\psi = g$, the algorithm returns the symbolic states whose zones satisfy g , in accordance with the symbolic interpretation of guards assumed in the statement. Now assume that the claim holds for all strict subformulas of ψ . If $\psi = \neg\psi_1$, then by the induction hypothesis, $\mathbf{Sat}(\psi_1) = \{s \in \mathcal{S} \mid \mathcal{G}, s \models \psi_1\}$. Algorithm 1 computes $\mathbf{Sat}(\neg\psi_1) = \mathcal{S} \setminus \mathbf{Sat}(\psi_1)$, which is exactly $\{s \in \mathcal{S} \mid \mathcal{G}, s \models \neg\psi_1\}$. If $\psi = \psi_1 \wedge \psi_2$, then by the induction hypothesis, $\mathbf{Sat}(\psi_i) = \{s \in \mathcal{S} \mid \mathcal{G}, s \models \psi_i\}$ ($i = 1, 2$). Algorithm 1 computes $\mathbf{Sat}(\psi_1 \wedge \psi_2) = \mathbf{Sat}(\psi_1) \cap \mathbf{Sat}(\psi_2)$, which coincides with the semantic interpretation of conjunction. If $\psi = j.\psi_1$, correctness follows from the assumed faithfulness of the symbolic reset operation with respect to the semantic clause of the freeze operator. It remains to consider the opacity-temporal cases: $\exists(\psi_1 \cup \psi_2)$, $\forall(\psi_1 \cup \psi_2)$, $\exists(\psi_1 \mathsf{R} \psi_2)$, $\forall(\psi_1 \mathsf{R} \psi_2)$. By assumption, the procedures $\mathbf{OpU}_{\exists}$, $\mathbf{OpU}_{\forall}$, $\mathbf{OpR}_{\exists}$, and $\mathbf{OpR}_{\forall}$ compute exactly the corresponding symbolic satisfaction sets. Hence the algorithmic clauses coincide with the semantic clauses in all four cases. Therefore, for every subformula $\psi \in \mathbf{Sub}(\varphi)$, $\mathbf{Sat}(\psi) = \{s \in \mathcal{S} \mid \mathcal{G}, s \models \psi\}$. Applying this to the full formula φ yields $\mathcal{G}, s_0 \models \varphi$ iff $s_0 \in \mathbf{Sat}(\varphi)$, which proves soundness and completeness.

Theorem 1 *The model-checking problem for ETOL over TA belongs to PSPACE.*

Proof. Let $\mathcal{A}$ be a timed automaton and let $\varphi \in \text{ETOL}$. The model-checking procedure works over the symbolic zone-graph abstraction of $\mathcal{A} \oplus x_0$, where the additional clock x_0 is never reset and records the total elapsed duration. Under the standard extrapolation operators, such as Extra_{LU}^+ [12, 7], the symbolic zone graph is finite. The algorithm evaluates φ bottom-up over its finite set of

Formula	Init. sat.	Time (s)
OE(T U takeCash)	false	0.002080
j.OA(!(MAQ MAN) U (E & j<=100))	false	0.005729
OE(T U j.(takeCash & j <= 20))	false	0.002380
OA((PQW & x<= 15) U j.(takeCash & j<= 5))	false	0.003565
x.((!C & x <= 100) & j.OE(correctPwd U (takeCash & j2 <= 30)))	false	0.002169
OA(!takeCash U j.(E & j =10))	false	0.002853

Table 1. ATM case study: execution time per ETOL formula and satisfaction in the initial state.

subformulas. Boolean cases are handled by standard symbolic operations on sets of symbolic states. The opacity-temporal cases are handled by the fixed-point procedures based on the predecessor operators $Pre_{\ominus}$ and $Pre_{\forall}$. Each symbolic state is represented by a location together with a zone, encoded for instance by a DBMs [18] representation, whose size is polynomial in the number of clocks and in the binary encoding of the constants appearing in $\mathcal{A}$ and φ . Although the complete symbolic zone graph may be exponentially large, it is not necessary to construct it explicitly. Its symbolic states and predecessor relations can be generated on-the-fly. At each stage of the computation, the procedure stores only: (i) the current subformula, (ii) a polynomial-size symbolic representation of the current state or zone, and (iii) polynomially many auxiliary variables used by the fixed-point computation. Therefore, the whole model-checking procedure can be carried out in polynomial space. Hence the model-checking problem for ETOL over TA belongs to PSPACE.

7 Model Checker for ETOL

We developed a symbolic model checker for the logic ETOL⁶, designed to verify timed opacity properties over timed systems modeled as TA. The verification algorithm is based on symbolic zone-based exploration, extended with support for reasoning about freeze clocks. The verification engine operates over a symbolic representation of time using DBMs, enabling efficient manipulation and comparison of clock valuations. The tool is implemented in Python 3.11 and covers the full core of ETOL, including freeze clocks and both variants of the execution-time opacity operator: *existential* opacity ($\ominus$) and *universal* opacity ($\forall$). Intuitively, $\ominus(\varphi)$ requires the existence of a time-matching alternative behaviour for at least one witness behaviour satisfying φ , whereas $\forall(\varphi)$ requires such time-matching alternatives for all relevant behaviours (according to the formal semantics of ETOL).

⁶ Source code available at: <https://github.com/jortizve/ETOL>

7.1 Algorithmic workflow.

Given a model and a formula, the checker proceeds as follows:

1. Symbolically explore reachable zones using DBMs [18], while maintaining the global non-resetting clock for execution duration.
2. Evaluate subformulas bottom-up on the zone graph.
3. For opacity formulas of the form $\exists \varphi$ or $\forall \varphi$, construct (when possible) pairs of timed traces with identical duration: a *witness trace* satisfying φ and an *alternative trace* of the same duration satisfying $\neg\varphi$.

This trace-pair construction is used to justify opacity (or to exhibit a counterexample when it fails).

7.2 Reproducible Evaluation by Scripts

The experimental evaluation is fully reproducible through a script-based workflow (no graphical interface is required). We used:

- `GeneratorATM.py`, which generates scalable ATM instances by extending a fixed ATM core with additional session modules, and
- `benchmark_total_mem_time.py`, which runs the model checker on a list of ETOL formulas and records execution-time and memory statistics (peak RSS and Python allocations), as well as the number of explored zones when available.

We executed the checker on multiple case studies, including the ATM scenario (detailed in example 1) and synthetic ATM-like families generated at increasing sizes. The remainder of this section reports representative results on the base ATM instance with larger-scale benchmarks and resource-usage plots.

7.3 ATM Case Study: Results

Table 1 reports the outcomes of verifying a set of representative ETOL formulas on the ATM model. For each formula, we provide the measured runtime and whether the formula holds in the initial state. The tool additionally outputs the set of satisfying states computed for the formula.

7.4 Example: Input file (ATM Model)

```
1 Transition
2 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
3 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
4 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0
5 1 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0
6 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
7 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
```

```

8 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
9 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0
10 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
11 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1
12 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
13 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
14 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
15 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
16 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
17 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
18
19 Name_State
20 I W WP WC WA PNW PQW DB MAN MAQ OO T C E
21
22 Initial_State
23 I
24
25 Atomic_propositions
26 cash balance correctPwd
27
28 Labelling
29 0 0 0
30 0 0 0
31 0 0 0
32 0 0 1
33 0 0 0
34 0 0 0
35 0 0 0
36 0 1 0
37 1 0 0
38 1 0 0
39 0 0 0
40 0 0 0
41 0 0 0
42 0 0 0
43
44 Number_of_agents
45 1
46
47 Clocks
48 x y
49
50 Clock_constraints
51 0 x:=0,y:=0 0 0 0 0 0 0 0 0 0 0 0
52 0 0 x=3,x:=0 0 0 0 0 0 0 0 0 0 0 0
53 0 0 0 x:=0 0 0 0 0 0 0 0 0 x=10 0
54 0 0 0 0 x:=0 0 x:=0 x:=0 0 0 0 0 0
55 0 0 0 0 0 x:=0 0 0 0 0 0 0 x=12 0
56 0 0 0 0 0 0 0 x=15,x:=0 0 0 0 0 0
57 0 0 0 0 0 0 0 0 x=15,x:=0 0 0 0 0

```

```

58 0 0 0 0 0 0 0 0 0 0 x=10,x:=0 0 0 0
59 0 0 0 0 0 0 0 0 0 0 x:=0 0 x=20 0
60 0 0 0 0 0 0 0 0 0 0 0 0 x=20 0
61 0 0 0 0 0 0 0 0 0 0 0 0 x=10 0 0
62 0 0 0 0 0 0 0 0 0 0 0 0 0 y=100
63 0 0 0 0 0 0 0 0 0 0 0 0 0 y=100
64 0 0 0 0 0 0 0 0 0 0 0 0 0 0
65
66 Invariants
67 0 0
68 x<=3 0
69 x<=10 0
70 x<=10 0
71 x<=10 0
72 x<=15 0
73 x<=15 0
74 x<=10 0
75 x<=20 0
76 x<=20 0
77 x<=10 0
78 0 y<=100
79 0 y<=100
80 0 0

```

Listing 1.1. ATM Input Model

7.5 Input ATM Properties

```

1
2 OE(T U takeCash)
3
4 j.OA(!(MAQ | MAN) U (E \& j<=100))
5
6 OE(T U j.(takeCash \& j <= 20)))
7
8 OA((PQW \& x<= 15) U j.(takeCash \& j <= 5))
9
10 j1.((!C \& j1 <= 100) \& j2.OE(correctPwd U (takeCash \& j2 <= 30)))
11
12 OA(!takeCash U j.(E \& j =10)

```

Listing 1.2. ATM ETOL Properties

The [Listing 1.1](#) shows a snippet of the ATM model in a text format compatible with ETOL. It defines the states, transitions, clocks, and invariants of the system. The [Listing 1.2](#) presents example ETOL properties that can be verified against this model, focusing on ET-opacity and timing constraints. For each property, the tool reports whether the initial state satisfies the formula,

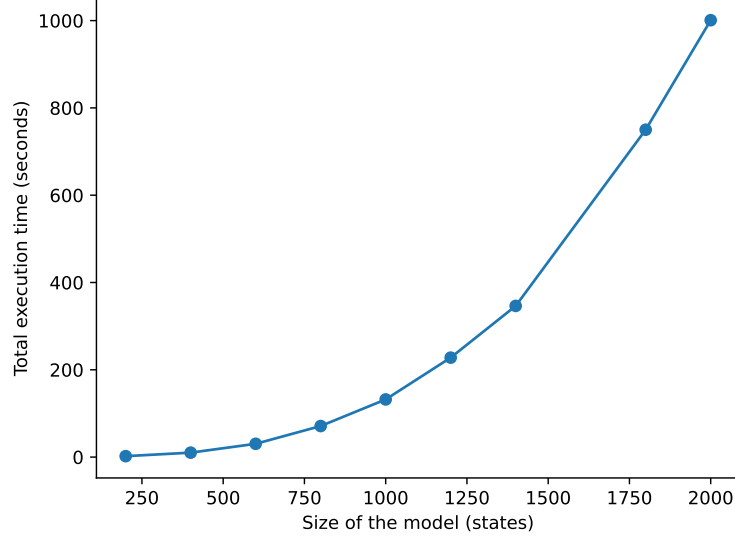


Fig. 3. Tool execution times relative to the model size.

and when applicable it outputs a witness execution together with an alternative execution of identical total duration. The tool completed the verification within 0.0285 seconds. To assess scalability, we conducted additional experiments on automatically generated TA. We generate multiple random instances and measure the verification time of each property. We report the average runtime over all generated instances, as well as standard deviation. Figure 3 plots execution time versus model size. In our synthetic ATM experiments, the analyzed models are not the handcrafted ATM instance of the case study: they are generated automatically by extending the ATM core with additional session modules (e.g., authentication, operation selection, and withdrawal/balance branches). This construction scales the system while preserving the semantics of the key control states and atomic propositions used in the opacity specifications. Model size is measured by the total number of states and transitions; atomic propositions on the added states are assigned automatically, while the original ATM labeling is kept unchanged, yielding diverse yet semantically consistent instances. As shown in Figure 3, execution times exhibit the expected polynomial trend with respect to model size. For each reported size, we generated and verified a large set of random ATM instances, and the figure reports average running time. The largest models include over 1,000 states and a large transition relation, yet our zone-based opacity checking remains efficient, completing verification within a few seconds on standard hardware.

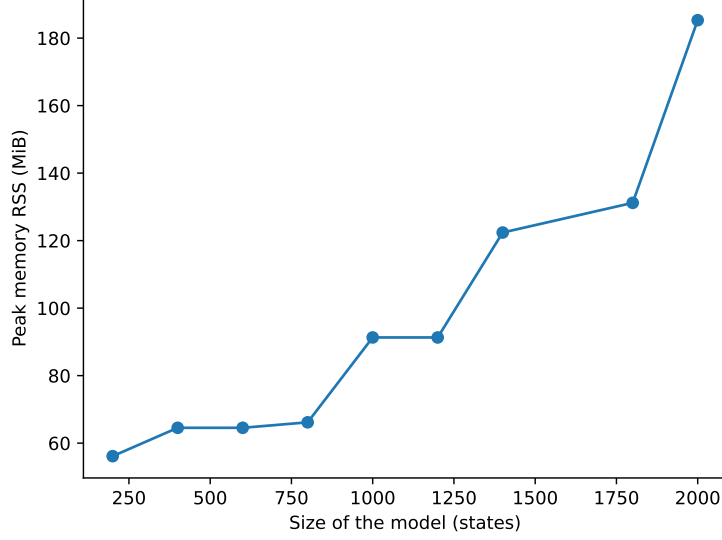


Fig. 4. Tool execution memory relative to the model size.

To complement runtime measurements, we report peak memory consumption during model checking using two indicators: (i) the peak resident set size (RSS), which captures the overall memory footprint of the process (including Python objects and native allocations), and (ii) the peak Python-allocated memory measured by `tracemalloc`, which reflects memory allocated by Python-level objects only. As shown in Fig. 4 and Fig. 5, both measures increase smoothly with the number of states, confirming a controlled growth of the explored symbolic state space. The RSS curve is consistently higher than the `tracemalloc` curve, as expected, since RSS also includes interpreter overhead, libraries, and non-Python allocations.

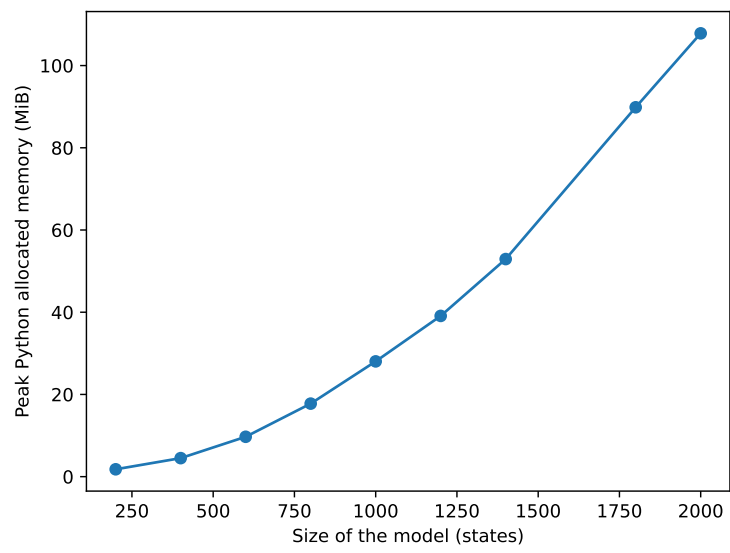


Fig. 5. Tool execution memory relative to the model size.